

Методическая разработка: подготовка Linux-сервера и публикация первого web-сервиса

1. Паспорт работы

Тема лекции: Linux-сервер предприятия и первый web-сервис.

Учебный сценарий: подготовить учебный Linux-сервер, назначить понятное имя и постоянный IPv4-адрес, установить web-сервер, опубликовать простую страницу, открыть доступ через firewall, проверить результат с сервера и клиентской машины, зафиксировать параметры в паспорте сервера.

ОС: Debian 12 Server или Ubuntu Server, Linux Client или Windows Client.

ПО: systemd, systemctl, apt, Nginx или Apache, ufw, ss, curl, journalctl, браузер.

Формат: демонстрация преподавателя и самостоятельное повторение студентами.

Время: 2-4 академических часа.

Предварительная подготовка: студенты уже умеют входить в Linux, выполнять команды с sudo, редактировать файлы через nano, проверять IP-адреса и работать с виртуальными машинами.

Итоговый результат: сервер web01 имеет постоянный IP-адрес, на нём работает web-служба, опубликована учебная HTML-страница, порт HTTP доступен с клиента, а студент может подтвердить настройку командами и записями в журнале.

2. Что будет настроено

В ходе реализации сценария настраивается базовый Linux-сервер предприятия:

- имя сервера web01;
- статический IPv4-адрес 192.168.10.20/24;
- обновлённый индекс пакетов;
- web-сервер Nginx как основной вариант;
- простая HTML-страница организации;
- правило firewall для HTTP;
- проверка службы через systemctl;
- проверка порта через ss;
- проверка web-страницы через curl и браузер;
- просмотр журналов web-сервера;
- паспорт сервера с параметрами и командами проверки.

3. Схема учебного стенда

Текстовая схема:

```
client01
192.168.10.101
  |
  | HTTP-запрос TCP/80
  v
web01.company.local
192.168.10.20
Linux Server + Nginx
```

Адресный план:

Узел	Роль	ОС	IP-адрес	Назначение
web01	Linux-сервер	Debian 12 Server / Ubuntu Server	192.168.10.20/24	Публикация web-страницы
client01	Клиент проверки	Linux или Windows	192.168.10.101/24	Проверка web-доступа

Параметры сервиса:

Параметр	Значение
Hostname сервера	web01
FQDN для будущих работ	web01.company.local
Web-сервер	Nginx
HTTP-порт	80/tcp
Каталог страницы Nginx	/var/www/html
Главный файл страницы	/var/www/html/index.html
Access log Nginx	/var/log/nginx/access.log
Error log Nginx	/var/log/nginx/error.log

4. Исходные условия и правила безопасности

Перед началом работы нужно проверить:

- серверная и клиентская ВМ находятся в одной виртуальной сети;
- у сервера есть доступ в интернет или к локальному репозиторию пакетов;
- у пользователя есть право выполнять команды через `sudo`;
- перед изменением сетевых настроек создан снимок ВМ;
- известны имя сетевого интерфейса и текущий IP-адрес.

Проверка текущих IP-адресов:

```
ip addr
```

Что делает команда:

- показывает сетевые интерфейсы;
- показывает IPv4 и IPv6-адреса;
- помогает найти имя интерфейса, например `enp0s3`, `ens33` или `eth0`.

Проверка маршрута по умолчанию:

```
ip route
```

Что делает команда:

- показывает маршруты Linux;
- помогает проверить, есть ли gateway для выхода в другие сети.

Проверка доступа к клиенту:

```
ping -c 4 192.168.10.101
```

Что делает команда:

- отправляет 4 ICMP-запроса клиентской машине;
- подтверждает базовую IP-связность до установки web-сервиса.

Важно: при изменении сетевой конфигурации можно потерять доступ к серверу. Перед применением Netplan или другого сетевого конфигуратора нужно проверить имя интерфейса и сохранить снимок ВМ.

5. Подготовка Linux-сервера

5.1. Проверка версии ОС и ядра

Сначала нужно понять, с какой системой работает администратор. От версии дистрибутива зависят имена пакетов, пути к файлам и способ настройки сети.

```
cat /etc/os-release
```

Что делает команда:

- выводит сведения о дистрибутиве;
- показывает имя ОС и версию.

Пример ожидаемого результата:

```
PRETTY_NAME="Debian GNU/Linux 12 (bookworm)"
```

Проверка версии ядра:

```
uname -r
```

Что делает команда:

- показывает версию Linux kernel;
- полезна при диагностике драйверов, сетевых адаптеров и виртуальных машин.

5.2. Назначение имени сервера

Сервер должен иметь понятное имя. Имя web01 сразу показывает роль узла: первый web-сервер.

```
sudo hostnamectl set-hostname web01
```

Что делает команда:

- устанавливает статическое имя сервера;
- записывает его в системную конфигурацию;
- помогает идентифицировать сервер в журналах, мониторинге и документации.

Проверка:

```
hostnamectl
```

Ожидаемый результат:

```
Static hostname: web01
```

5.3. Настройка локального имени в /etc/hosts

Пока DNS ещё не настроен, сервер может сопоставлять своё имя с IP-адресом через файл /etc/hosts.

Открыть файл:

```
sudo nano /etc/hosts
```

Пример содержимого:

```
127.0.0.1    localhost
192.168.10.20 web01.company.local web01
```

Что означает запись:

- 192.168.10.20 — IP-адрес сервера;
- web01.company.local — полное доменное имя;
- web01 — короткое имя.

Проверка:

```
getent hosts web01
```

Ожидаемый результат:

```
192.168.10.20    web01.company.local web01
```

6. Настройка статического IPv4-адреса

6.1. Почему серверу нужен статический адрес

Клиентские машины могут получать адреса автоматически, но серверные службы должны быть доступны по постоянному адресу. Если IP-адрес web-сервера изменится, клиенты, DNS-записи, firewall-правила и документация перестанут соответствовать реальности.

В этой работе используется адрес:

```
192.168.10.20/24
```

Параметры:

Параметр	Значение
IP-адрес	192.168.10.20
Маска	/24 или 255.255.255.0
Сеть	192.168.10.0/24
Gateway	192.168.10.1, если в стенде есть маршрутизатор
DNS	192.168.10.10, если DNS уже есть, или временно внешний DNS

6.2. Настройка через Netplan

В Ubuntu Server часто используется Netplan. Имя файла может отличаться, например /etc/netplan/00-installer-config.yaml или /etc/netplan/01-netcfg.yaml.

Посмотреть файлы Netplan:

```
ls /etc/netplan
```

Что делает команда:

- показывает YAML-файлы сетевой конфигурации;
- помогает выбрать файл для редактирования.

Открыть файл:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

Пример настройки:

```
network:
  version: 2
  ethernets:
    enp0s3:
      addresses:
        - 192.168.10.20/24
      routes:
        - to: default
          via: 192.168.10.1
      nameservers:
        addresses:
          - 192.168.10.10
          - 8.8.8.8
```

Что важно:

- `enp0s3` нужно заменить на имя интерфейса из `ip addr`;
- `addresses` задаёт статический IP-адрес;
- `routes` задаёт шлюз по умолчанию;
- `nameservers` задаёт DNS-серверы.

Проверка конфигурации перед применением:

```
sudo netplan try
```

Что делает команда:

- временно применяет сетевые настройки;
- даёт возможность откатить изменения, если связь пропала.

Применить настройки:

```
sudo netplan apply
```

Проверка:

```
ip addr show enp0s3  
ip route
```

Ожидаемый результат:

- интерфейс имеет адрес `192.168.10.20/24`;
- в маршрутах есть `default route` через `192.168.10.1`, если шлюз используется.

6.3. Если используется Debian с `/etc/network/interfaces`

В некоторых установках Debian сеть настраивается через файл `/etc/network/interfaces`.

Открыть файл:

```
sudo nano /etc/network/interfaces
```

Пример настройки:

```
auto enp0s3  
iface enp0s3 inet static  
    address 192.168.10.20  
    netmask 255.255.255.0  
    gateway 192.168.10.1  
    dns-nameservers 192.168.10.10 8.8.8.8
```

Перезапуск сетевой службы зависит от дистрибутива. В учебной VM проще перезагрузить сервер после проверки файла:

```
sudo reboot
```

Что делает команда:

- перезагружает сервер;
- применяет сетевую конфигурацию при следующем запуске.

Проверка после загрузки:

```
ip addr  
ip route
```

7. Обновление пакетов

Перед установкой службы нужно обновить индекс пакетов. Индекс пакетов — это локальный список доступных пакетов и версий из подключённых репозиториев.

```
sudo apt update
```

Что делает команда:

- обращается к репозиториям;
- обновляет список доступных пакетов;
- не устанавливает и не обновляет сами программы.

Пример ожидаемого результата:

```
Reading package lists... Done
```

Если появляются ошибки сети или DNS, нужно проверить:

```
ping -c 4 8.8.8.8  
ping -c 4 deb.debian.org
```

Что показывают проверки:

- успешный ping до 8.8.8.8 подтверждает IP-доступ в интернет;
- успешный ping до deb.debian.org подтверждает работу DNS.

8. Установка web-сервера

8.1. Выбор Nginx или Apache

Для первой работы достаточно одного web-сервера. В методичке основной вариант — Nginx.

Nginx часто используют как:

- web-сервер для статических страниц;
- reverse proxy;
- точку входа к нескольким внутренним сервисам;
- балансировщик на базовом уровне.

Apache HTTP Server часто используют как:

- классический web-сервер;
- платформу с развитой модульной системой;
- сервер для приложений, которым нужны .htaccess и специфические модули.

В этой работе используется Nginx, потому что он прост для первой демонстрации: пакет, служба, порт, страница, журнал.

8.2. Установка Nginx

```
sudo apt install nginx -y
```

Что делает команда:

- устанавливает пакет nginx;
- добавляет systemd-службу nginx;
- создаёт стандартный каталог /var/www/html;
- создаёт журналы в /var/log/nginx.

Проверка установленного пакета:

```
dpkg -l nginx
```

Что делает команда:

- показывает состояние установленного пакета;
- строка с `ii` означает, что пакет установлен.

Пример ожидаемого результата:

```
ii  nginx  1.22.1-9  amd64  small, powerful, scalable web/proxy server
```

8.3. Альтернативный вариант: Apache

Если преподаватель выбирает Apache, установка выполняется так:

```
sudo apt install apache2 -y
```

Что делает команда:

- устанавливает Apache HTTP Server;
- создаёт службу `apache2`;
- использует каталог `/var/www/html`;
- создаёт журналы в `/var/log/apache2`.

Проверка:

```
systemctl status apache2
```

В дальнейших шагах для Apache вместо `nginx` используется `apache2`.

9. Управление web-службой через `systemctl`

9.1. Проверка состояния службы

`systemctl` управляет службами `systemd`. Для администратора важно не просто установить пакет, а доказать, что служба запущена и готова принимать запросы.

```
systemctl status nginx
```

Что делает команда:

- показывает состояние службы `Nginx`;
- показывает, загружен ли `unit`-файл;
- показывает, запущен ли процесс;
- показывает последние строки журнала.

Ожидаемый результат:

```
Active: active (running)
```

9.2. Запуск, остановка и перезапуск

Запустить службу:

```
sudo systemctl start nginx
```

Что делает команда:

- запускает службу в текущей сессии;
- не обязательно включает автозапуск после перезагрузки.

Перезапустить службу:

```
sudo systemctl restart nginx
```

Что делает команда:

- останавливает и снова запускает службу;

- применяется после серьёзных изменений конфигурации.

Мягко перечитать конфигурацию:

```
sudo systemctl reload nginx
```

Что делает команда:

- заставляет Nginx перечитать конфигурацию без полного перезапуска;
- обычно используется после изменения сайта или виртуального хоста.

Включить автозапуск:

```
sudo systemctl enable nginx
```

Что делает команда:

- добавляет службу в автозагрузку;
- после перезагрузки сервера Nginx стартует автоматически.

Проверка автозапуска:

```
systemctl is-enabled nginx
```

Ожидаемый результат:

```
enabled
```

10. Проверка порта web-сервера

Web-сервер считается доступным только если его процесс слушает нужный сетевой порт. Для HTTP используется порт 80/tcp.

```
sudo ss -tulpen | grep ':80'
```

Что делает команда:

- ss показывает сетевые сокеты;
- -t показывает TCP;
- -u показывает UDP;
- -l показывает только слушающие порты;
- -p показывает процесс;
- -e показывает расширенную информацию;
- -n выводит адреса и порты числами;
- grep ':80' оставляет строки с HTTP-портом.

Ожидаемый результат:

```
tcp LISTEN 0 511 0.0.0.0:80 0.0.0.0:* users:(("nginx",pid=...,fd=...))
```

Как читать результат:

- LISTEN означает, что порт принимает подключения;
- 0.0.0.0:80 означает, что служба слушает порт 80 на всех IPv4-интерфейсах;
- nginx показывает процесс, который слушает порт.

Если строки нет, проверить:

```
systemctl status nginx  
journalctl -u nginx -n 50 --no-pager
```


11. Публикация простой HTML-страницы

11.1. Где находится страница

В стандартной установке Nginx основной каталог сайта:

```
/var/www/html
```

Главная страница:

```
/var/www/html/index.html
```

Когда клиент открывает `http://192.168.10.20`, web-сервер отдаёт файл `index.html`.

11.2. Создание учебной страницы

Открыть файл:

```
sudo nano /var/www/html/index.html
```

Записать содержимое:

```
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>web01</title>
</head>
<body>
  <h1>Linux web-сервер web01 работает</h1>
  <p>Служба Nginx опубликована в учебной сети.</p>
  <p>IP-адрес сервера: 192.168.10.20</p>
</body>
</html>
```

Что важно:

- файл должен называться `index.html`;
- файл должен находиться в `/var/www/html`;
- web-сервер должен иметь право читать этот файл.

Проверка файла:

```
ls -l /var/www/html/index.html
```

Ожидаемый результат:

```
-rw-r--r-- 1 root root ... /var/www/html/index.html
```

Права `rw-r--r--` означают:

- владелец может читать и изменять файл;
- группа и остальные пользователи могут читать файл;
- Nginx сможет отдать страницу клиенту.

12. Проверка web-сервиса на сервере

12.1. Локальная проверка через curl

`curl` — это инструмент для выполнения HTTP-запросов из командной строки. Он позволяет проверить web-сервер без браузера.

```
curl http://127.0.0.1
```

Что делает команда:

- отправляет HTTP-запрос на локальный адрес сервера;
- проверяет, отвечает ли web-служба на самом сервере.

Ожидаемый результат:

```
<h1>Linux web-сервер web01 работает</h1>
```

Проверка по IP-адресу сервера:

```
curl http://192.168.10.20
```

Что делает команда:

- проверяет доступ к web-серверу через его сетевой адрес;
- помогает отличить локальную работу службы от сетевой доступности.

12.2. Проверка HTTP-заголовков

```
curl -I http://192.168.10.20
```

Что делает команда:

- ключ -I запрашивает только HTTP-заголовки;
- быстро показывает статус ответа без вывода всей страницы.

Ожидаемый результат:

```
HTTP/1.1 200 OK  
Server: nginx
```

Как читать результат:

- 200 OK означает, что страница найдена и успешно отдана;
- Server: nginx подтверждает, что отвечает Nginx.

13. Настройка firewall

13.1. Проверка состояния UFW

Firewall ограничивает сетевой доступ к серверу. Даже если Nginx слушает порт 80, клиент может не подключиться, если firewall блокирует HTTP.

```
sudo ufw status verbose
```

Что делает команда:

- показывает состояние UFW;
- показывает разрешённые и запрещённые правила;
- помогает понять, открыт ли HTTP.

Если UFW не установлен:

```
sudo apt install ufw -y
```

Что делает команда:

- устанавливает простой инструмент управления firewall;
- подходит для учебной демонстрации базовых правил.

13.2. Разрешение SSH перед включением firewall

Если работа идёт по SSH, сначала нужно разрешить SSH, иначе можно потерять удалённый доступ.

```
sudo ufw allow OpenSSH
```

Что делает команда:

- разрешает входящие подключения к SSH;
- защищает от случайной потери доступа при включении firewall.

13.3. Разрешение HTTP

```
sudo ufw allow 80/tcp
```

Что делает команда:

- разрешает входящие TCP-подключения на порт 80;
- позволяет клиентам открывать web-страницу.

Можно использовать профиль приложения:

```
sudo ufw allow 'Nginx HTTP'
```

Что делает команда:

- разрешает HTTP по профилю Nginx;
- удобно, если пакет Nginx добавил профиль в UFW.

Включить firewall:

```
sudo ufw enable
```

Что делает команда:

- активирует UFW;
- применяет разрешающие и запрещающие правила.

Проверка:

```
sudo ufw status numbered
```

Ожидаемый результат:

80/tcp	ALLOW	IN	Anywhere
OpenSSH	ALLOW	IN	Anywhere

14. Проверка с клиентской машины

14.1. Проверка IP-связности

На клиенте Linux:

```
ping -c 4 192.168.10.20
```

На Windows-клиенте:

```
ping 192.168.10.20
```

Что делает команда:

- проверяет базовую сетевую доступность сервера;
- не доказывает работу web-сервера, но помогает исключить проблему IP-сети.

14.2. Проверка HTTP через браузер

На клиенте открыть:

```
http://192.168.10.20
```

Ожидаемый результат: отображается страница с заголовком:

```
Linux web-сервер web01 работает
```

Если страница не открывается, проверить по цепочке:

1. IP-адрес сервера.
2. Связь через ping.
3. Состояние службы Nginx.
4. Порт 80 через ss.
5. Правила firewall.
6. Журналы web-сервера.

14.3. Проверка HTTP с Linux-клиента через curl

```
curl -I http://192.168.10.20
```

Ожидаемый результат:

```
HTTP/1.1 200 OK
```

Если клиент получает Connection refused, это часто означает, что порт не слушается.

Если клиент получает timeout, это часто означает проблему маршрута, firewall или виртуальной сети.

15. Журналы web-сервера

15.1. Access log

Access log фиксирует успешные и неуспешные обращения клиентов к web-серверу.

Для Nginx:

```
sudo tail -n 20 /var/log/nginx/access.log
```

Что делает команда:

- показывает последние 20 строк access log;
- позволяет увидеть IP-адрес клиента, запрошенный ресурс и HTTP-статус.

Пример строки:

```
192.168.10.101 - - [16/Jul/2026:10:15:21 +0500] "GET / HTTP/1.1" 200 245 "-" "Mozilla/5.0"
```

Как читать строку:

- 192.168.10.101 — IP-адрес клиента;
- GET / — запрос главной страницы;
- 200 — успешный HTTP-ответ;
- Mozilla/5.0 — user-agent браузера.

15.2. Error log

Error log помогает искать проблемы web-сервера.

```
sudo tail -n 20 /var/log/nginx/error.log
```

Что делает команда:

- показывает последние ошибки Nginx;
- помогает найти проблемы прав, конфигурации или недоступных файлов.

Если ошибок нет, файл может быть пустым или содержать старые записи.

15.3. Журнал systemd-службы

```
journalctl -u nginx -b --no-pager
```

Что делает команда:

- показывает журнал службы nginx;
- ключ -u выбирает службу;
- ключ -b ограничивает вывод текущей загрузкой;
- --no-pager выводит текст сразу в терминал.

Этот журнал полезен, если Nginx не запускается или не пересчитывает конфигурацию.

16. Документирование параметров сервера

Администратор должен не только выполнить настройку, но и оставить понятную документацию. Без документации следующий специалист не поймёт, какие службы работают на сервере и как проверять результат.

16.1. Паспорт сервера

Создать файл паспорта:

```
sudo nano /root/server-passport-web01.md
```

Пример содержимого:

```
# Паспорт сервера web01
```

```
## Основные параметры
```

```
- Hostname: web01
- Роль: учебный web-сервер
- ОС: Debian 12 Server
- IP-адрес: 192.168.10.20/24
- Web-сервер: Nginx
- HTTP-порт: 80/tcp
```

```
## Команды проверки
```

```
- systemctl status nginx
- sudo ss -tulpen | grep ':80'
- curl -I http://192.168.10.20
- sudo tail -n 20 /var/log/nginx/access.log
- sudo ufw status numbered
```

```
## Результат
```

Страница доступна с client01 по адресу http://192.168.10.20.

Что важно:

- паспорт должен быть кратким;
- в нём должны быть реальные параметры стенда;
- команды проверки должны позволять быстро подтвердить работоспособность.

16.2. Проверка паспорта

```
sudo cat /root/server-passport-web01.md
```

Что делает команда:

- выводит созданный паспорт;
- позволяет убедиться, что параметры записаны без ошибок.

17. Диагностика и типовые ошибки

Диагностику web-сервиса нужно вести по цепочке: сеть, служба, порт, firewall, HTTP, журналы.

17.1. Базовая цепочка проверки

Проверить IP-адрес:

```
ip addr
```

Проверить маршрут:

```
ip route
```

Проверить службу:

```
systemctl status nginx
```

Проверить порт:

```
sudo ss -tulpen | grep ':80'
```

Проверить firewall:

```
sudo ufw status numbered
```

Проверить HTTP локально:

```
curl -I http://127.0.0.1
```

Проверить HTTP по сетевому адресу:

```
curl -I http://192.168.10.20
```

Проверить журнал:

```
sudo tail -n 20 /var/log/nginx/error.log
```

17.2. Типовые ошибки

Симптом	Возможная причина	Проверка	Исправление
Клиент не пингует сервер curl http://127.0.0.1 не отвечает	Неверная сеть VM или IP-адрес Nginx не запущен	ip addr, ping systemctl status nginx	Проверить адаптеры VM и адреса Запустить sudo systemctl start nginx
Порт 80 не слушается	Служба остановлена или ошибка конфигурации	ss, journalctl -u nginx	Исправить ошибку и перезапустить службу
Локально работает, с клиента нет	Firewall блокирует HTTP	sudo ufw status	Разрешить sudo ufw allow 80/tcp

Симптом	Возможная причина	Проверка	Исправление
Открывается стандартная страница, а не учебная	Не изменён index.html	cat /var/www/html/index.html	Записать нужный HTML-файл
Ошибка 403 Forbidden	Неверные права на файл или каталог	ls -l /var/www/html	Исправить права чтения
Ошибка 404 Not Found	Файл отсутствует или неверный путь	ls /var/www/html	Создать index.html

18. Итоговая самостоятельная проверка

Студент должен подтвердить результат по чек-листу:

- ☐ Сервер имеет имя web01.
- ☐ Сервер имеет статический IP-адрес 192.168.10.20/24.
- ☐ Клиент находится в той же учебной сети.
- ☐ Пакет Nginx установлен.
- ☐ Служба nginx находится в состоянии active (running).
- ☐ Порт 80/tcp слушается процессом Nginx.
- ☐ В firewall разрешён HTTP.
- ☐ Файл /var/www/html/index.html содержит учебную страницу.
- ☐ curl http://127.0.0.1 возвращает HTML-страницу.
- ☐ curl -I http://192.168.10.20 возвращает 200 OK.
- ☐ Клиент открывает страницу в браузере.
- ☐ В access.log появилась строка обращения клиента.
- ☐ Паспорт сервера заполнен.

Финальный набор команд проверки на сервере:

```
hostnamectl
ip addr
systemctl status nginx
sudo ss -tulpen | grep ':80'
sudo ufw status numbered
curl -I http://127.0.0.1
curl -I http://192.168.10.20
sudo tail -n 20 /var/log/nginx/access.log
```

19. Отчёт по работе

В отчёт включить:

1. Тему работы.
2. Схему учебного стенда.
3. Таблицу адресов сервера и клиента.
4. Скриншот или текстовый вывод hostnamectl.
5. Скриншот или текстовый вывод ip addr.
6. Команду установки web-сервера.
7. Результат systemctl status nginx.
8. Результат ss -tulpen | grep ':80'.
9. Содержимое опубликованной HTML-страницы.
10. Результат проверки curl -I.
11. Скриншот страницы в браузере клиента.
12. Фрагмент access.log с обращением клиента.
13. Паспорт сервера.
14. Описание одной найденной ошибки и способа её исправления.

Шаблон вывода:

```
В ходе работы был подготовлен Linux-сервер web01.  
Сервер получил статический IP-адрес 192.168.10.20/24.  
На сервер установлен и запущен Nginx.  
Порт 80/tcp открыт и доступен клиентской машине.  
Работа web-сервиса подтверждена командами systemctl, ss, curl и записью в access.log.
```

20. Контрольные вопросы

1. Почему серверу нужен постоянный IP-адрес?
2. Для чего администратор задаёт понятный hostname?
3. Как связаны пакет, служба, порт и журнал?
4. Чем проверка `systemctl status nginx` отличается от проверки `curl http://127.0.0.1`?
5. Почему открытый порт через `ss` ещё не гарантирует доступ с клиента?
6. Для чего нужен `firewall` при публикации web-сервиса?
7. Как определить, что клиент действительно обращался к web-серверу?
8. Какие сведения должны быть в паспорте сервера?
9. Что означает HTTP-статус 200 OK?
10. Что проверять, если локально web-сервер работает, а с клиента страница не открывается?

21. Итог

После выполнения методички студент должен понимать базовую логику сопровождения Linux-сервера: сервер сначала получает понятное имя и постоянный адрес, затем на него устанавливается служба, после этого проверяются состояние службы, порт, `firewall`, клиентский доступ и журналы. Настройка считается завершённой только тогда, когда результат подтверждён командами и записан в отчёт.